Sirona

# Documentation Plan

| STATUS | OWNER | LAST UPDATED |
| --- | --- | --- |
| Proposed | Sirona product owner | November 4, 2026 |

| | |
| --- | --- |
| **Authors** | Prepared with Codex |
| **Reviewers** | Owner and engineering reviewers |
| **Related docs** | Detailed Markdown plan in docs/engineering |
| **Scope** | Feature specifications and implementation handoffs for the local creation platform |

## 1. Abstract

Extend Sirona into a governed creation platform through connected, source-backed specifications. The orchestrator owns goals, plans, permissions and completion. Providers and coding workers remain replaceable. Start with an approved plan and a verified local application.

This review proposes the documentation architecture. The companion Markdown contains the full folder tree, 24 feature cards, build order, risks and coverage map. No product code, data migration, paid inference or deployment is authorized by this document.

## 2. Goals and Non-Goals

| Goals | Non-goals |
| --- | --- |
| One clear reading path from AGENTS to implementation evidence | A backend rewrite or new competing architecture |
| Four complete documents for each in-scope feature | Empty folders and placeholder implementations |
| Source-backed contracts, dependencies and acceptance tests | Unqualified provider, worker or sandbox promises |
| Preserve working code, user data, UI and approval history | Cloud operations or enterprise SaaS in the local MVP |

## 3. Background and Problem Statement

Nine feature bundles already exist, but source paths and status claims need reconciliation. The roadmap labels P5/P6 complete while active gates remain. Database documents disagree. The intent requirements service is a stub; the CLI currently offers read-only diagnostics.

Keep the monorepo and existing owners. Root instructions route agents to governance, package instructions and the feature index. Each feature links requirements to contracts, source files, tests and evidence. Documentation quality does not establish runtime readiness.[1]

[1]Source inspection establishes code presence only. No fresh product acceptance is claimed in this review.

## 4. Proposed Architecture

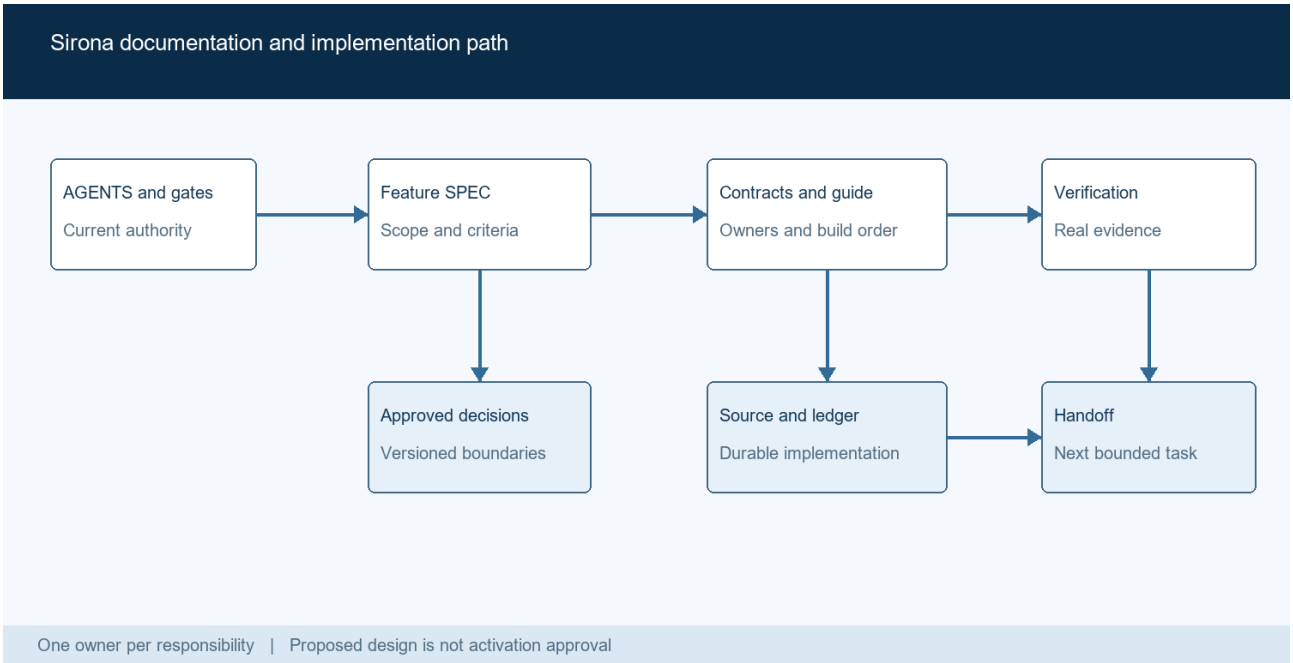



Figure 1. Proposed documentation reading and evidence path.

### Core components

| Component | Responsibility | Primary storage | Failure behavior |
|---|---|---|---|
| Governance | Own current decisions, scope and approval gates. | docs/governance | Conflicts block affected implementation. |
| Architecture | Define shared boundaries and system invariants. | docs/architecture | No feature may invent a competing owner. |
| Feature bundles | Specify behavior, contracts, build steps and proof. | docs/features | Missing prerequisites remain explicit. |
| Execution ledger | Track tasks, dependencies and evidence references. | docs/engineering | Unverified work cannot become complete. |
| Reference and proof | Catalog actual interfaces and sanitized results. | docs/reference and evidence | Stale evidence is not current qualification. |

## 5. Request Lifecycle

a. Start with the current user task, root instructions and active gates.

b. Read package ownership and the relevant feature SPEC; confirm scope.
c. Load contracts and direct prerequisite evidence, not every archive.
d. Prepare a bounded task with exact source owners, changes and tests.
e. Obtain any required approval before migrations or protected effects.
f. Implement the complete slice; enforce budgets, cancellation and recovery.
g. Run real acceptance, record limitations and update the owning docs.

## 6. API and Data Contracts

### Proposed feature documentation metadata

| Field | Type | Required | Description |
|---|---|---|---|
| featureId | String | Yes | Stable identity used by tasks, criteria and evidence. |
| designStatus | Enum | Yes | Proposed, approved or superseded; never inferred from tests. |
| requirements | ID list | Yes | Measurable behavior, boundaries and acceptance criteria. |
| sourceOwners | Path list | Yes | Editable files and symbols responsible for the workflow. |
| dependencies | ID list | Yes | Feature prerequisites and active approval gates. |
| verification | Records | Yes | Criterion, command, source revision, target and result. |
| activation | Record | Yes | Separate production enablement and owner approval state. |

### Contract guarantees

a. An approved design is not proof that a feature is implemented.
b. Implementation progress, verification and activation are separate states.
c. Evidence names the source revision, environment, criterion and limitations.
d. Generated catalogs describe interfaces; they cannot grant authority.

Canonical wire types stay in packages/contracts; feature prose links to them.

See the companion plan sections 6, 9, 10 and 13 for the full contract rules.

## 7. Consistency, Idempotency, and Replay

Preserve one durable owner per responsibility. Runs pin approved revisions; effects keep stable identities across attempts. Reconcile uncertain outcomes before retrying. The chosen database must prove these semantics; common parameter syntax is not sufficient.

| Scenario | Expected behavior | Reasoning |
|---|---|---|
| Repeated action | Return or reconcile the recorded effect result. | Avoid duplicate writes and external operations. |
| Required write fails | Stop dispatch and report a safe explicit error. | Do not execute an effect without durable authority. |
| Unknown tool outcome | Reconcile first; retry only the approved class. | A timeout does not prove the action never happened. |
| Contract changes | Keep the run pinned; propose a new revision. | Old approval cannot authorize changed scope. |

## 8. Security and Privacy Considerations

a. Enforce local identity, workspace scope and exact action approval in trusted code.
b. Treat repository, browser, memory and worker content as untrusted input.
c. Resolve secret references through the credential broker; exclude keys from evidence.
d. Retain data, image and paid-inference gates. A timeout is not an OS sandbox.
e. Define retention, correction and deletion; preserve provenance without private payloads.

## 9. Operational Readiness

| Signal | SLO or alert | Owner | Launch gate |
|---|---|---|---|
| Requirement coverage | All in-scope requirements have criteria and owners. | Feature owner | Required |
| Documentation links | Active links, source paths and anchors resolve. | Docs owner | Required |
| Qualification gaps | No unknown criterion represented as passed. | Verifier | Required |

| Signal | SLO or alert | Owner | Launch gate |
|---|---|---|---|
| Authority drift | Database, phase and approval conflicts reconciled. | Product owner | Required |
| User workflow | Real UI or CLI journey and recovery evidence. | Feature owner | Required |

Rollout constraint: documentation completion permits review of a bounded task. Product activation still requires qualified dependencies, real acceptance and the applicable owner gates.

## 10. Alternatives Considered

| Alternative | Why it was considered | Why it was not selected |
|---|---|---|
| One giant master prompt | Easy to write and share initially. | Hard to maintain; hides ownership and conflicts. |
| Copy another product | Established documentation patterns. | Its architecture and permissions do not define Sirona. |
| Many small services | Apparent domain independence. | Adds operational complexity before demonstrated need. |
| Folders without contracts | Fast visible scaffolding. | Agents still guess behavior, failures and acceptance. |

## 11. Open Questions

a. Reconcile the recorded SQLite-only versus dual-engine architecture decision.
b. Approve an exact provider, model and spend ceiling before live inference.
c. Review the exact tools image before guest qualification and activation.
d. Choose one supported worker and deployment target for later releases.

## 12. Decision and Next Steps

Adopt the documentation plan as a proposal, then start with the source and authority audit. Prove the format on providers, transport and CLI before expanding the remaining features. Follow the existing product gates; do not treat this plan as implementation approval.

| Milestone | Deliverable | Exit criteria |
| --- | --- | --- |
| M1 | D01-D03 audit, navigation and four-file templates | Conflicts explicit; source owners and status rules clear. |
| M2 | D04 pilot provider, gateway and CLI bundles | Complete journeys, contracts and verification paths. |
| M3 | D05-D07 remaining features and coverage | Every product requirement has a release and owner. |
| M4 | D08-D09 validation and cold handoff | An agent can prepare a bounded task without guessing. |